

Improved Recipe for Compress-then-Train

Yequan Zhao

June 10, 2026

Goal: a better recipe for *compress-then-train*: optimize for the model *after* training, not only at compression time.

- **Reasoning-aware compression:** preserve long-sequence generation (reasoning) capability
- **Training-aware compression** SVD, BTT and Nystrom for MoE expert compression
- **Extended training recipe:** use **on-policy distillation (OPD)** to keep improving on reasoning tasks after standard KD plateaus.

Reasoning-aware compression: calibration preserves generation

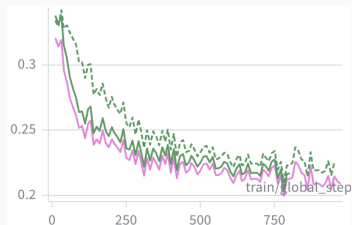
Challenge. Structured pruning/compression was very weak at *reasoning*

Cause. The calibration covariances were collected *truncated fixed-length windows* — losing long chain of thoughts transition, the compressed model doesn't know where to stop.

Fix. Calibrate on the *full reasoning trace* (no truncation) and *reweight by sequence* (every conversation counts equally, not every token).

Result. Improvement on both training-free, and the **advantage holds through the training.**

calibration scheme	MATH-500	C4 PPL
2048 windows	56.0%	103.0
full sequence	69.0%	98.7
full sequence + reweight	71.0%	92.4



Input-output whitened SVD

$$\hat{\mathbf{W}} = \arg \min_{\hat{\mathbf{W}}} \left\| \mathbf{C}_g^{1/2} (\mathbf{W} - \hat{\mathbf{W}}) \mathbf{C}_x^{1/2} \right\|_F^2,$$

solved by truncated SVD of $\mathbf{M} = \mathbf{C}_g^{1/2} \mathbf{W} \mathbf{C}_x^{1/2}$, with $\mathbf{C}_x = \sum_t \mathbf{x}_t \mathbf{x}_t^\top$ (forward activations) and $\mathbf{C}_g = \sum_t \mathbf{g}_t \mathbf{g}_t^\top$ (back-propagated loss gradients).

This two-sided whitening is a Hessian estimate for the layer weights:

$$\mathbf{H}_{\mathbf{W}} = \mathbb{E} \left[\mathbf{x} \mathbf{x}^\top \otimes \mathbf{g}_y \mathbf{g}_y^\top \right] \approx \mathbb{E} \left[\mathbf{x} \mathbf{x}^\top \right] \otimes \left[\mathbf{g}_y \mathbf{g}_y^\top \right].$$

So truncation **discards the least loss-sensitive subspace**, and those surviving high-curvature directions are exactly the ones training keeps actively updating.

Nyström: build a single joint forward+backward PSD kernel

$\mathbf{K}_{\text{joint}} = \mathbf{C}_f^{1/2} \mathbf{C}_b \mathbf{C}_f^{1/2}$ and select the hidden neurons by Nyström

approximation: keeping the directions important to *both* the forward pass and the gradient that trains $\mathbf{W}_u, \mathbf{W}_g$.

OBD-LLM derivation: closed-form loss-aware decomposition

Objective. Second-order Taylor expansion of the task-loss change from decomposing $\mathbf{W} \rightarrow \hat{\mathbf{W}} = \mathbf{B}\mathbf{A}$:

$$\Delta L(\mathbf{w}) \approx (\mathbf{w} - \hat{\mathbf{w}}) \mathbf{H}_{\mathbf{w}} (\mathbf{w} - \hat{\mathbf{w}})^\top.$$

K-FAC factorization. Approximate $\mathbf{H}_{\mathbf{w}} = \mathbf{C}_x \otimes \mathbf{C}_g$, with $\mathbf{C}_x = \mathbb{E}[\mathbf{X}\mathbf{X}^\top] = \mathbf{L}_x \mathbf{L}_x^\top$ and $\mathbf{C}_g = \mathbb{E}[\mathbf{G}\mathbf{G}^\top] = \mathbf{L}_g \mathbf{L}_g^\top$ (Cholesky).

Rewrite the loss. With $\Delta \mathbf{W} = \mathbf{W} - \hat{\mathbf{W}}$,

$$\begin{aligned} \Delta L(\mathbf{W}) &= \text{vec}(\Delta \mathbf{W}) (\mathbf{C}_x \otimes \mathbf{C}_g) \text{vec}(\Delta \mathbf{W})^\top \\ &= \text{vec}(\Delta \mathbf{W}) \text{vec}(\mathbf{C}_g \Delta \mathbf{W} \mathbf{C}_x)^\top \\ &= \text{tr}(\Delta \mathbf{W}^\top \mathbf{C}_g \Delta \mathbf{W} \mathbf{C}_x) = \left\| \mathbf{L}_g^\top \Delta \mathbf{W} \mathbf{L}_x \right\|_F^2. \end{aligned}$$

Solution. A truncated SVD of the whitened weight $\tilde{\mathbf{W}} = \mathbf{L}_g^\top \mathbf{W} \mathbf{L}_x$ minimizes the Frobenius norm, hence **minimizes the loss change** ΔL — a closed-form optimal decomposition.

MoE expert compression: doubly-whitened Nyström

OLMoE-1B-7B-Instruct, compression ratio 0.5(2×); After-training = short SFT recovery (~10k OpenThoughts math samples).

method	storage/active	training-free		after training	
		MMLU	GSM8K	MMLU	GSM8K
original (uncompressed)	1.0/1.0	0.542	0.	—	—
Nyström train-aware	0.5/0.5	0.382	0.150	0.379	0.215
Nyström	0.5/0.5	0.341	0.135	0.360	0.185
REAP (expert removal)	0.5/1.0	0.289	0.000	0.325	0.050
SlimQwen (expert merge)	0.5/1.0	0.269	0.005		

- expert compression > expert removal/merge
- Nyström training-aware has improvement on both training-free and after training.

Training recipe: off-policy KD vs. on-policy distillation

- **Off-policy KD** — let student match the teacher's traces.
- **On-policy distillation (OPD)** — student *generates* the responses, teacher *scores* them token-by-token. The student is corrected exactly where *it* goes wrong.

Two regimes, in order:

- **Stage 1** — **off-policy KD is more efficient.** Cheap dense supervision recovers most of the quality fast.
- **Stage 2** — **when Stage 1 plateaus, OPD keeps improving** on reasoning tasks

- Integrate the training-aware compression and demonstrate in the Percipio training pipeline
- Progressive compression: C_g captures the curvature of uncompressed model, but the curvature changes after compression
- Expert rank/c.f. allocation: Nystrom to minimize the error for the whole MoE module.
- On-policy distillation when current KD plateaus